Universidad de Monterrey

Facultad de Ingeniería

Proyecto de introducción a la ingeniería

Título:

A2.4 Equipo 7

Profesor: Antonio Martínez Torteya
Alumnos:
Ines Castillo Enríquez, 692934
Julián Sabino Duque Saldívar, 691784
Karol Alessandra Sandoval Lara, 650421
Emiliano García Vidales, 692086

“Doy mi palabra que he realizado esta actividad con integridad académica”

Introducción

El objetivo de este proyecto fue poder prender los motores y que estos al momento de ir hacia atrás emitan un sonido al tener objetos cerca gracias a un sensor de distancia que emite un sonido. Esto con el objetivo de poder crear un sistema que funcione como una ambulancia que es parte de los trabajos que estamos realizando que nos ayudarán en nuestro proyecto final.

Metodología

I. Conexiones del sistema

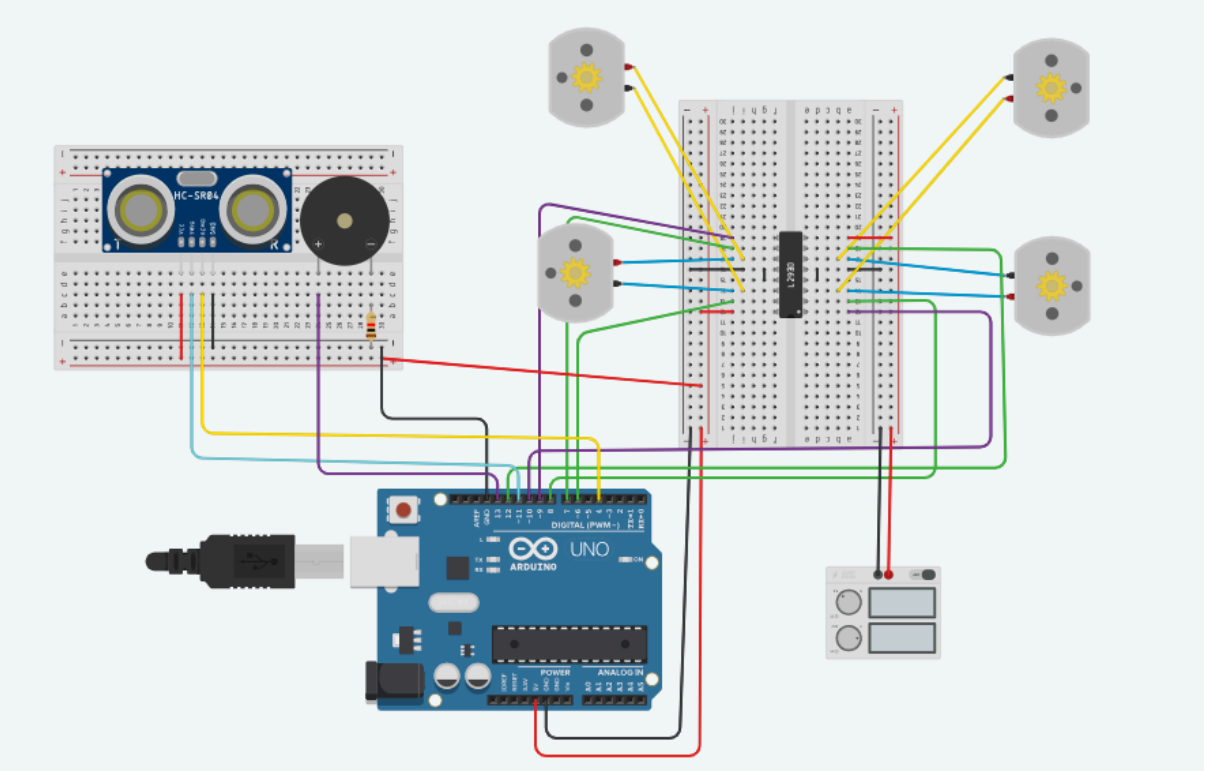


Figura 1: Componentes del sistema en TinkerCAD

Descripción y Justificación de Componentes

1. **Motores CC:** Se utilizan para transformar la energía eléctrica en energía mecánica, generando un movimiento giratorio. En un circuito, permiten mover ruedas, engranajes u otros mecanismos.

2. **Arduino:** Es una placa electrónica programable que funciona como el cerebro del circuito. Recibe información de sensores o botones y controla componentes como los motores y el piezoeléctrico mediante instrucciones programadas. Aquí es donde metemos el código para el circuito.
3. **Fuente de alimentación:** Se utiliza para suministrar la energía eléctrica necesaria para que los componentes del circuito funcionen correctamente.
4. **Piezo:** Se utiliza para generar sonidos o señales acústicas cuando recibe una señal eléctrica. Puede funcionar como una alarma o emitir sonidos para indicar que ocurrió un evento determinado dentro del circuito. En nuestro caso lo utilizaremos para que este genere un sonido cuando el sensor reconozca un objeto cercano.
5. **Sensor de distancia ultrasónico:** Sirve para que el sistema pueda detectar si está despejado al frente o no mediante lanzamientos constantes de sonido el sensor cuenta cuanto se tardo en regresar el sonido esto para poder identificar mientra menos tiempo tarde el sonido en regresar más cerca está el objeto.
6. **Control de motor:** Se utiliza para regular el funcionamiento de un motor, permitiendo controlar aspectos como su velocidad, sentido de giro y encendido o apagado. En un circuito con Arduino, permite controlar el motor mediante señales eléctricas, ya que el Arduino por sí solo generalmente no proporciona la corriente suficiente para alimentar un motor directamente.

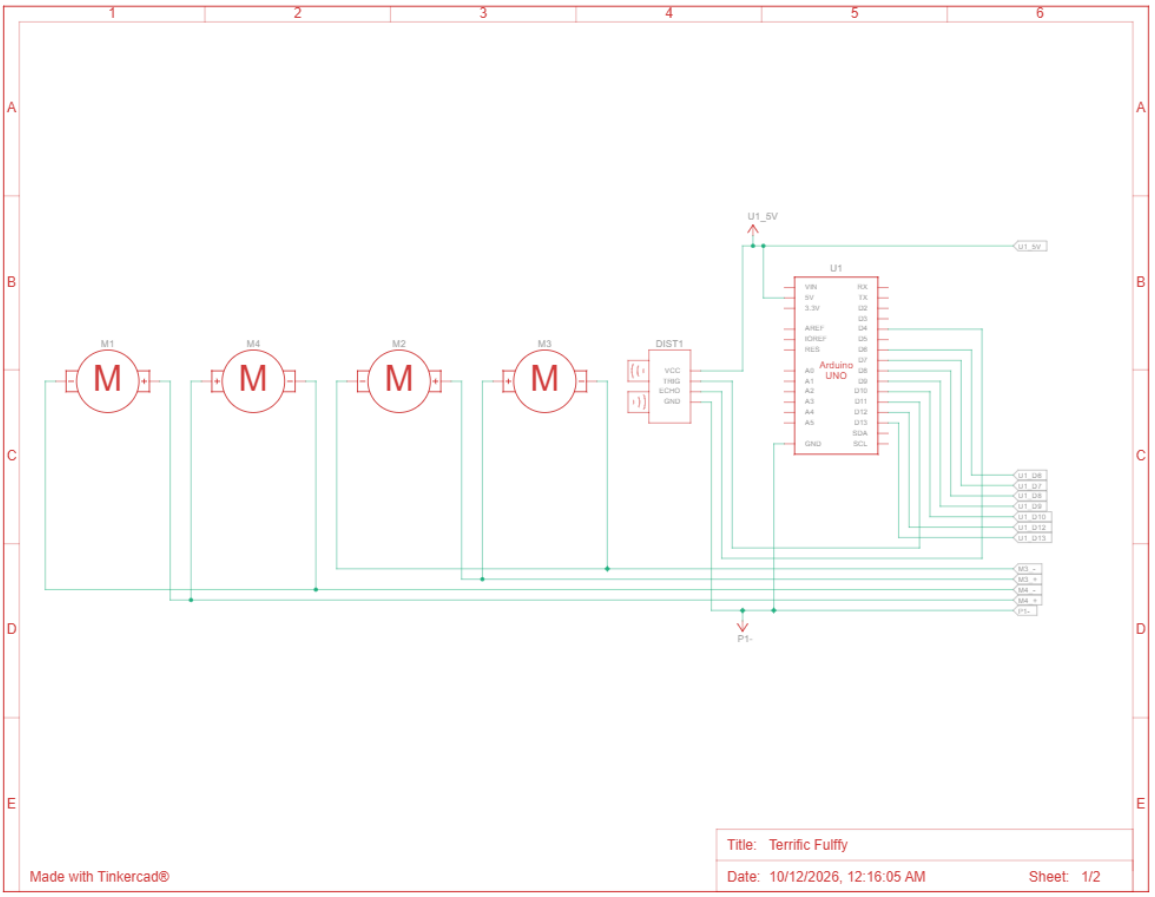


Figura 2: Diagrama de las conexiones del sistema generado por TinkerCAD

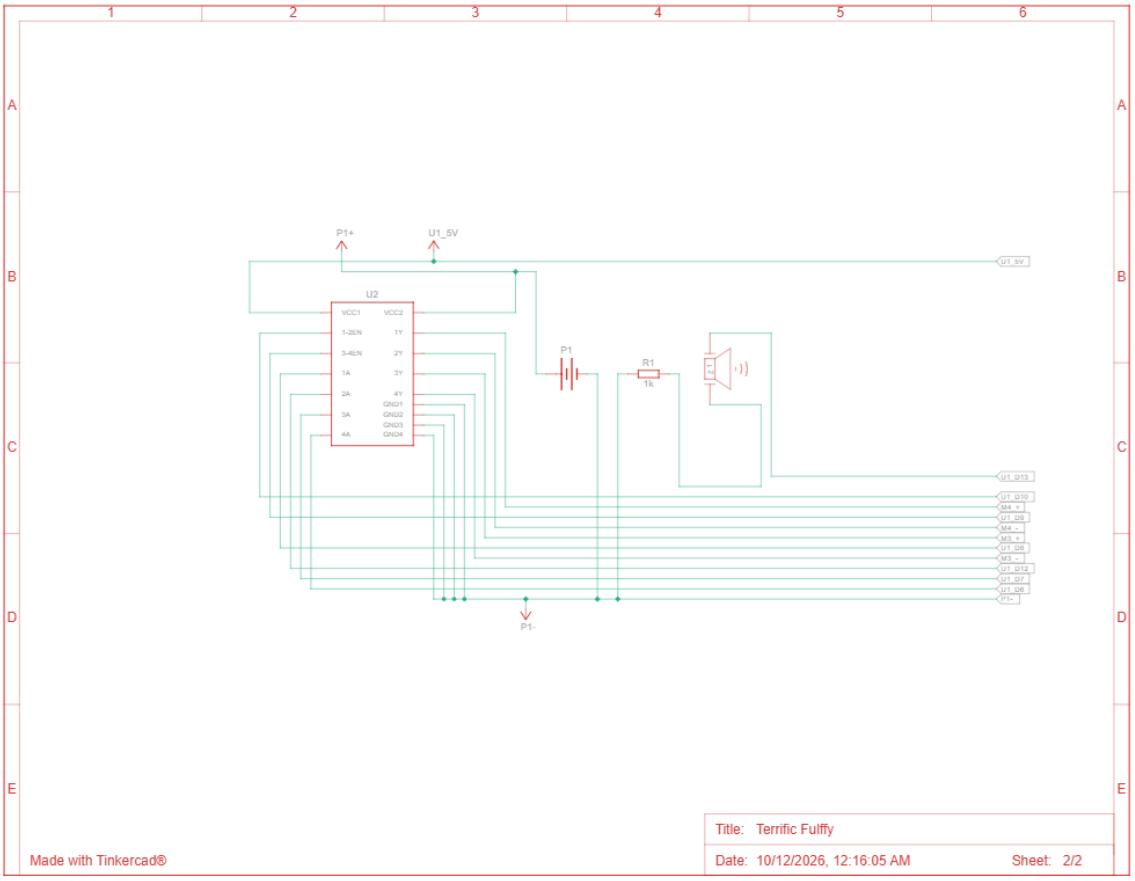


Figura 2: Diagrama de las conexiones del sistema generado por TinkerCAD

1. Ensamblaje y Justificación de Nodos

1. **Tierra común:** El puente H L293D, el sensor ultrasónico, el zumbador y la fuente externa comparten el mismo riel negativo continuo en las tarjetas de prueba. Justificación: Establece un nivel de referencia de voltaje estrictamente uniforme (0V).

2. **Aislamiento de voltaje:** El Arduino suministra 5V exclusivamente para las operaciones lógicas del puente y sensor L293D-H. En contraste, una fuente externa solo suministra energía a los motores, evitando que sus picos de alta corriente e interferencia inductiva se reinicien o dañen el microcontrolador.

Nivel de potencia y cinemática

1. **Transmisión lateral paralela:** Los motores izquierdo y derecho están agrupados paralelos a sus respectivas salidas del puente en H. Esto

permite un sistema de control de frenos que le permite girar hacia adelante, hacia atrás o alrededor de su propio eje con menos pines.

2. **Velocidad y Dirección (PWM):** Los pines PWM del Arduino están conectados a los habilitadores del L293D para regular de manera independiente la velocidad de cada flanco, mientras que los pines digitales estándar determinan la dirección de rotación de las llantas.

Sistema de percepción y alarma

1. **Sensor ultrasónico (HC-SR04):** Utiliza un pin (disparador) para emitir pulsos de sonido y otro (eco) para recibir el rebote. El microcontrolador calcula la distancia de los obstáculos basándose en el tiempo entre las dos señales.

2. **Alarm Buzzer:** Funciona como alarma de proximidad. Integra una resistencia en serie con tierra para limitar el amperaje y proteger el puerto digital del Arduino de posibles cortocircuitos o sobrecargas.

II. Código

```
// ===== SENSOR ULTRASONICO Y BUZZER =====
const int trigPin = 11;
const int echoPin = 4;
const int buzzerPin = 13; // (Usa un buzzer pasivo para que pued

// ===== DRIVER LADO IZQUIERDO =====
const int IN1 = 6;
const int IN2 = 7;
const int ENA = 9;

// ===== DRIVER LADO DERECHO =====
const int IN3 = 8;
const int IN4 = 12;
const int ENB = 10;

// Velocidad de los motores
const int velocidad = 200;

void setup() {
  // Sensor y Buzzer
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(buzzerPin, OUTPUT);

  // Motores
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);

  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);

```

```

  detener();
  delay(1000); // Pequeña pausa inicial de seguridad
}
```

Figura 3: Configuración Inicial del Sistema

Como se muestra en la figura 3, primero se definieron múltiples constantes de tipo entero que representan los pines a los que están conectados los motores y el sensor ultrasónico. Asimismo, se definió una constante que almacenará el valor de la velocidad de los motores.

Posteriormente, dentro de la función “void setup()” del código, se configuraron los pines de los motores y el buzzer como salida, ya que por medio de ellos el Arduino los controlará. Además, se configuraron los pines del sensor: el de “Trigger” como salida y el de “Echo” como entrada. Finalmente, se utilizó la función “detener()” para apagar todos los motores y se incluyó una pausa de un segundo antes de continuar con el resto del código.

```
38 void loop() {
39   // 1. Medir la distancia actual del sensor
40   long distancia = medirDistancia();
41
42   // 2. Todos los motores irán hacia atrás todo el tiempo
43   atras();
44
45   // 3. Evaluar la proximidad del objeto mientras retrocede
46   if (distancia > 0 && distancia <= 90) {
47     // Si hay un objeto a 90 cm o menos, el buzzer suena con inte
48     int frecuencia = map(distancia, 1, 90, 2000, 400);
49     tone(buzzerPin, frecuencia);
50   }
51   else {
52     // Si está a más de 90 cm o no hay obstáculos cercanos, el bu
53     noTone(buzzerPin);
54   }
55 }
56
57 // ===== FUNCIÓN PARA MEDIR DISTANCIA (EN CM) =====
58 long medirDistancia() {
59   digitalWrite(trigPin, LOW);
60   delayMicroseconds(2);
61   digitalWrite(trigPin, HIGH);
62   delayMicroseconds(10);
63   digitalWrite(trigPin, LOW);
64
65   long duracion = pulseIn(echoPin, HIGH, 30000); // Timeout de 30
66   long distanciaCm = duracion * 0.034 / 2;
67   return distanciaCm;
68 }
69
```

Figura 4: Lógica de Ejecución (Loop y Validación)

En cuanto a la función “void loop()”, primero se guardaron los valores que entrega la función “medirDistancia()” dentro de una variable y se usó la función “atras()” debido a que es necesario que los motores se dirijan hacia atrás en todo momento. Después, mediante una estructura de selección, se estableció que si la distancia entre el objeto y el sensor es mayor a 0 y menor o igual a 90 cm, se transformará el valor de la distancia a una frecuencia de 2000-400 Hz, con la cual el buzzer emitirá un sonido cada vez más fuerte conforme el objeto se acerque al sensor. De lo contrario, el buzzer no generará sonido.

Finalmente, se escribieron los bloques de código correspondientes a las funciones utilizadas. En primer lugar, dentro de “medirDistancia()” se mide la proximidad del objeto enviando un pulso y midiendo el tiempo que el pin Echo se mantiene en “HIGH”, valor con el cual se calcula la distancia.

```
69
70  // ===== ATRÁS (Sincronizado para que los 4 motores retrocedan re
71  void atras() {
72    // Lado izquierdo
73    digitalWrite(IN1, HIGH);
74    digitalWrite(IN2, LOW);
75
76    // Lado derecho (invertido respecto al izquierdo para empujar h
77    digitalWrite(IN3, LOW);
78    digitalWrite(IN4, HIGH);
79
80    analogWrite(ENA, velocidad);
81    analogWrite(ENB, velocidad);
82  }
83
84  // ===== DETENER =====
85  void detener() {
86    analogWrite(ENA, 0);
87    analogWrite(ENB, 0);
88    digitalWrite(IN1, LOW);
89    digitalWrite(IN2, LOW);
90    digitalWrite(IN3, LOW);
91    digitalWrite(IN4, LOW);
92  }
```

Figura 5: Fin de código

En segundo lugar, la función “atras()” determina el sentido de giro de los motores, cuyas combinaciones de “HIGH” y “LOW” están invertidas entre sí para que se dirijan hacia atrás. Por último, la función “detener()” establece en 0 las señales PWM y coloca los pines de los motores en “LOW”, deteniendo el giro de los motores.

III. Resultados

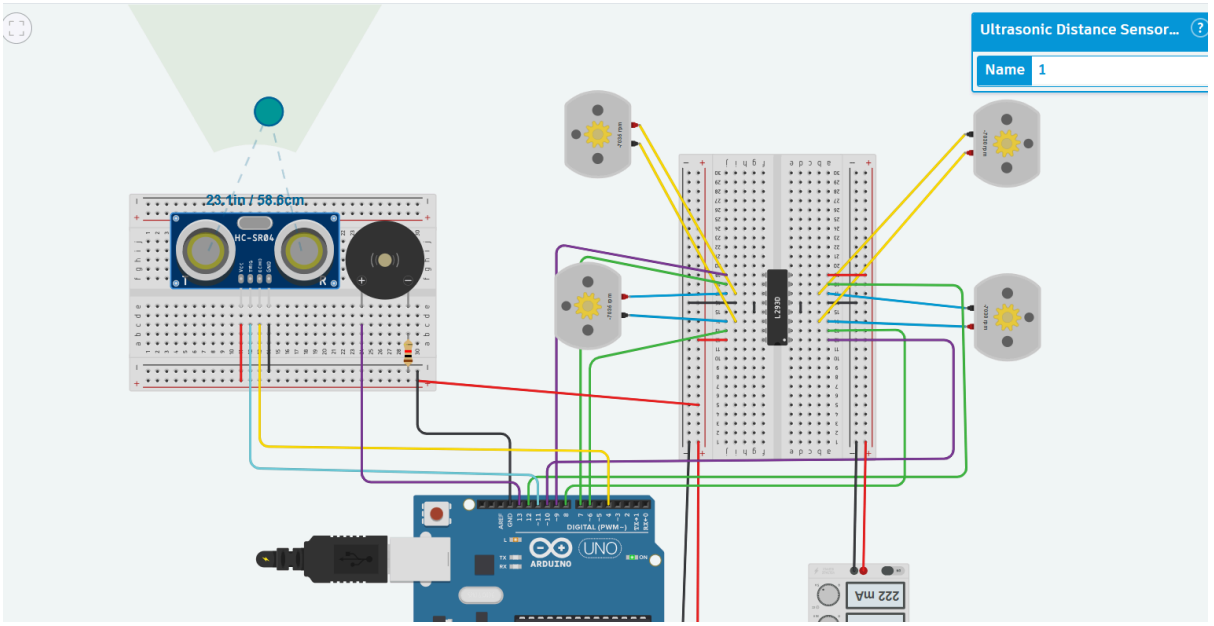


Figura 8: Circuito con componentes físicos

Cuando el motor va de reversa y detecta un objeto cercano en la reversa pite constantemente y mientras este se acerca al objeto el sonido suena más fuerte.

IV. Referencias

- Gemini, se utilizó el prompt:
  “Necesito que, utilizando como referencia este código, o modifiques para que el circuito ahora esté automatizado, sin que se presione el botón para tener que iniciar, además, necesito que ahora los motores estén retrocediendo todo el tiempo y, por último, quiero que, cuando el sensor detecte que un objeto está a 90 cm de él, el buzzer empezará a sonar y, mientras más cerca esté el objeto (es decir, menos centímetros), el buzzer empiece a sonar más fuerte.